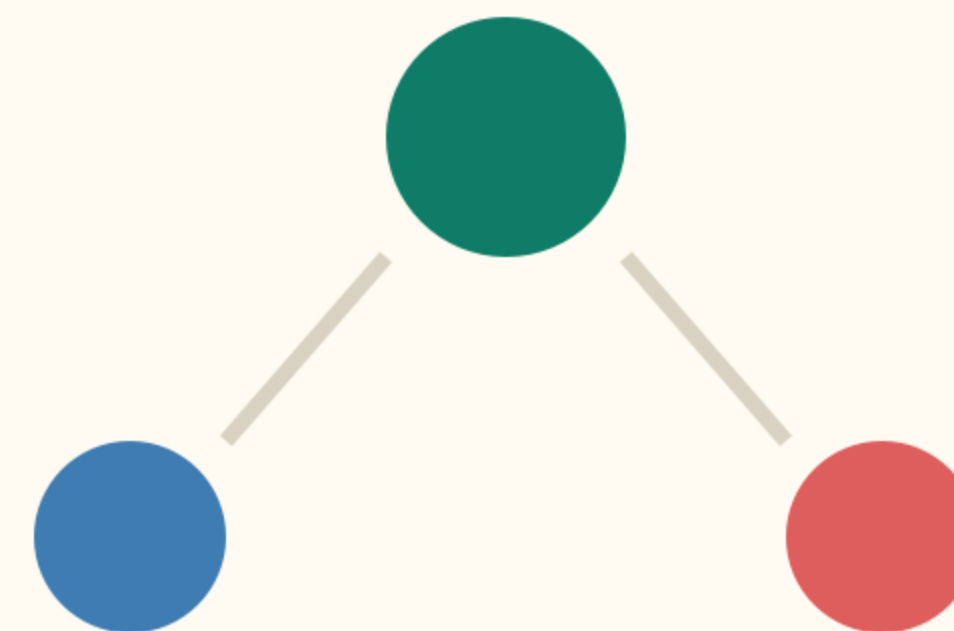




CSP-J 初赛 · 概念专题课

数据结构的奇妙世界

指针 · 链表 · 树 · 二叉树 · 图 —— 初赛必考概念一讲通



本讲内容

指针 · 链表 · 树 · 二叉树 · 图的概念、性质、存储与遍历

01 指针与链表

指针的本质是内存地址；链表用指针把分散的节点串联起来。



02 树与二叉树

一对多的层级结构：树的术语、二叉树性质与四种遍历。



03 图

顶点与边构成的关系模型：分类、存储与遍历。



04 考点速记

本讲核心公式与结论一页汇总，方便考前复习。



01

第 1 章 · CHAPTER 01

指针与链表

内存地址与指针的本质，以及由指针串联的链式结构。



内存如同旅馆：地址就是门牌号

内存示意：10 个存储单元，每个单元有唯一地址



变量 **a = 42** 被存入内存时，系统为它分配了 **地址 1006** 的存储单元。

只要知道地址，就能直接访问该变量，无需逐个单元查找。

- 内存按字节划分为一个个存储单元
- 地址：每个存储单元的唯一编号
- 数据存放在地址对应的存储单元中

★ 核心结论

程序中的每个变量都占有一块内存空间，其位置由地址唯一确定。

指针：存放内存地址的变量



$\&a \rightarrow 1001$ $\&$ 取地址运算符

$*p \rightarrow 42$ $*$ 解引用运算符

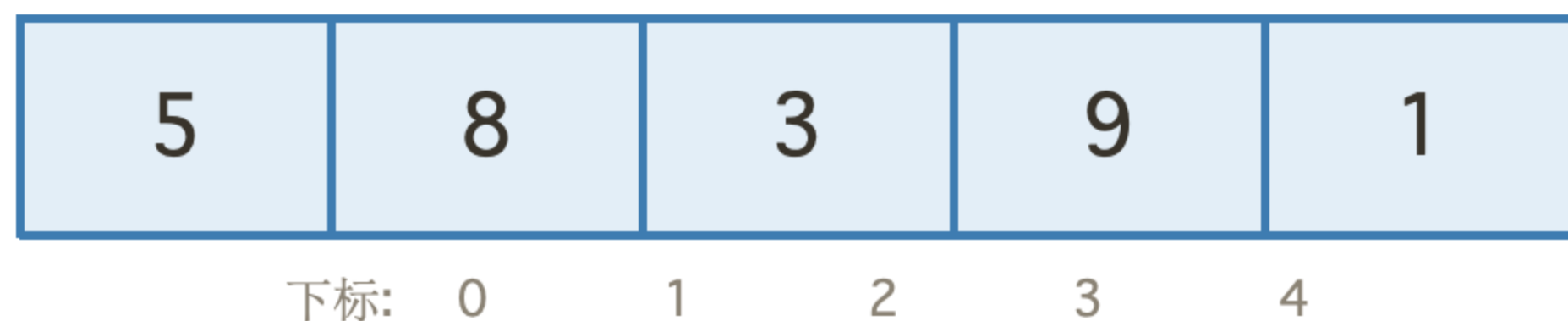
● 指针是一种变量，它的值是另一个对象的内存地址。

● 指针有类型：**int*** 表示指向 **int** 的指针，决定读取的字节数。

★ 初赛常考： $\&$ 取地址， $*$ 解引用；数组名在表达式中通常退化为指向首元素的指针。

数组 vs 链表：连续存储 vs 链式存储

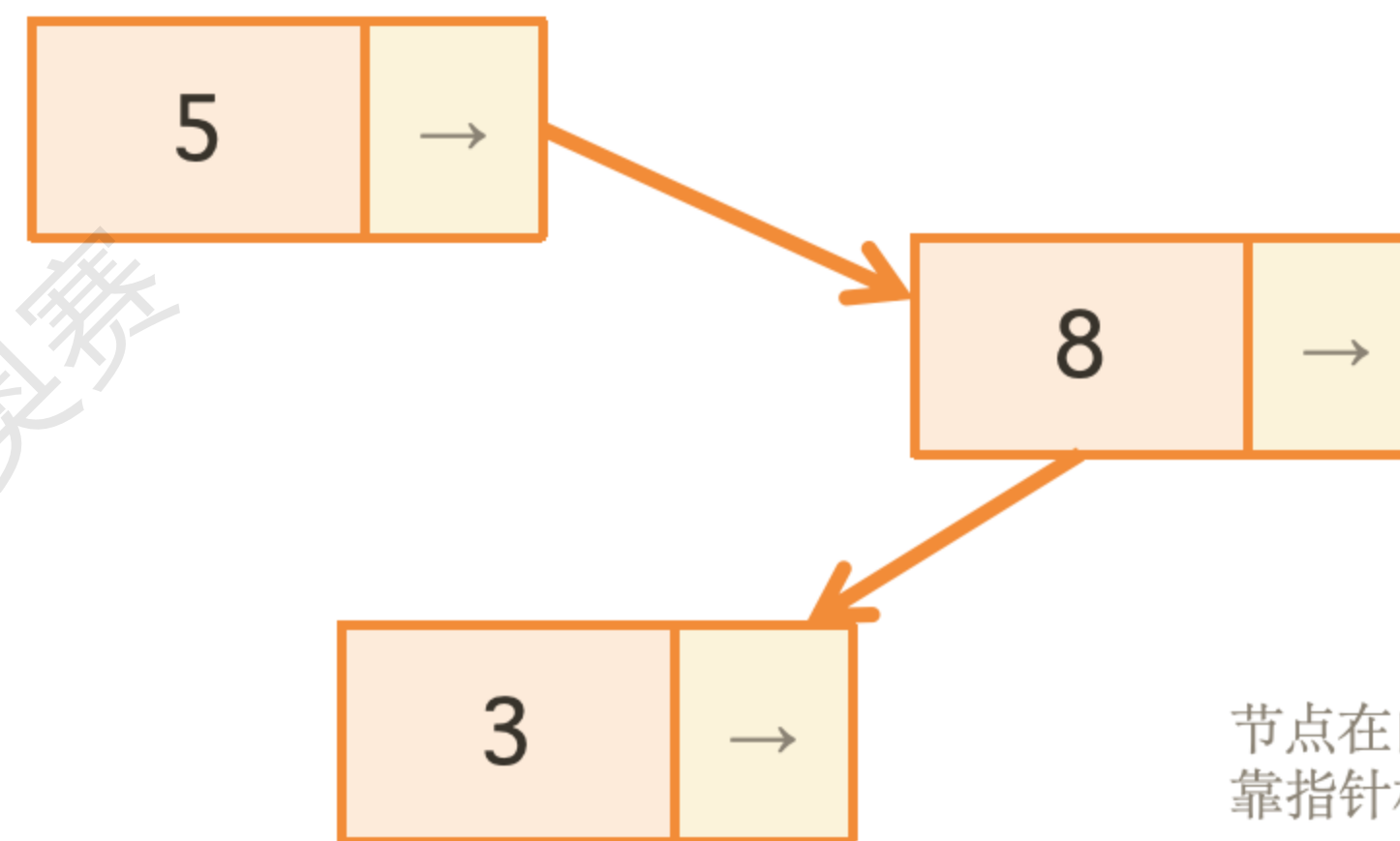
数组：连续存储



↑
在下标 1 处插入：后续元素需整体后移

- ✓ 下标随机访问，一步定位： $O(1)$
- ✗ 中间插入/删除需移动后续元素： $O(n)$

链表：链式存储



- ✓ 插入/删除只需修改指针： $O(1)$
- ✗ 访问第 k 个元素需从头遍历： $O(n)$

对比结论：数组支持 $O(1)$ 随机访问，但插入/删除需移动元素；链表插入/删除只需修改指针，但只能顺序访问。

单链表的结构：数据域 + 指针域



节点 = 数据域 (保存元素值) + 指针域 (保存后继节点的地址)。

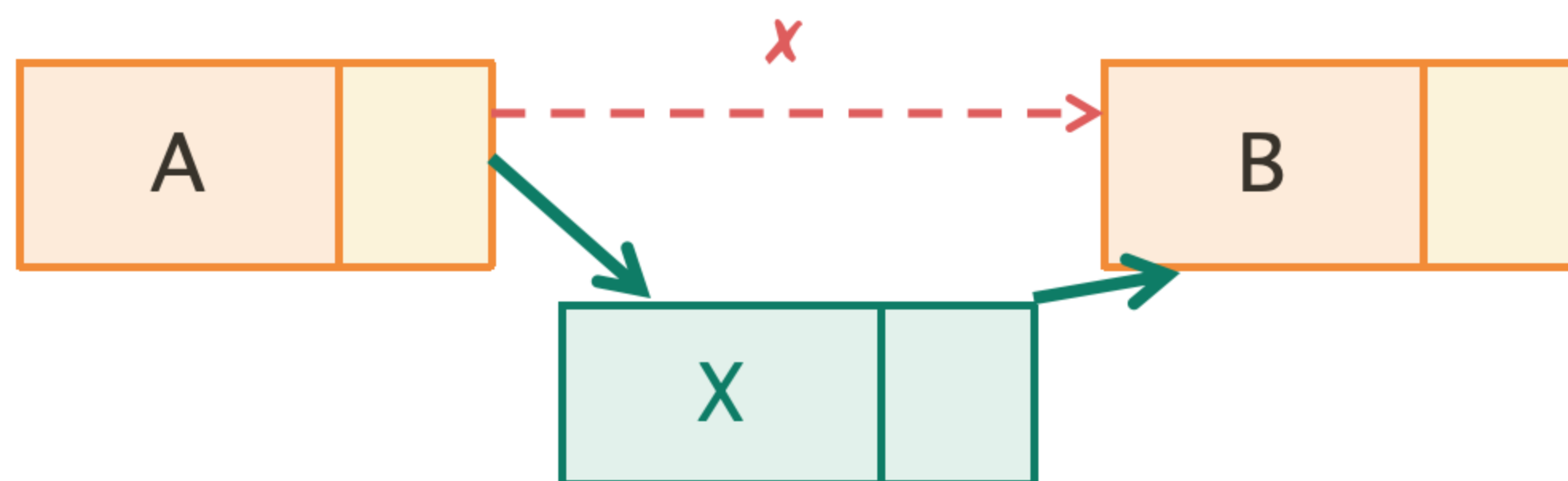
尾节点的指针域为 **NULL** (空指针)，表示链表到此结束。

● 单链表只能从头指针出发沿后继方向遍历，无法逆向。

● 访问元素 9 需从 head 顺序遍历：7 → 3 → 9，时间为 $O(n)$ 。

★ 初赛常考：链表节点在内存中不连续，依靠指针连接；头指针是访问整条链表的唯一入口。

① 插入：在 A 与 B 之间插入新节点 X



两个步骤（绿色箭头）：

① 令 X 的指针域指向 B

② 将 A 的指针域改为指向 X

原 A→B 链接失效，其余节点不必移动

② 删除：删除节点 B



一个步骤：

① 将 A 的指针域改为 B 的后继（即 C 的地址）

② 不再有任何指针指向 B，B 即从链表中移除

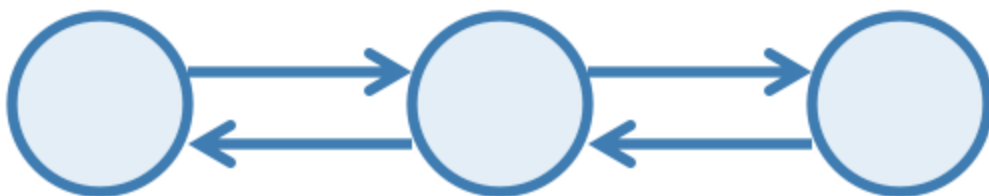
灰色 B：等待释放的节点

单向链表



只能沿后继方向遍历；结构最简单，应用最广。

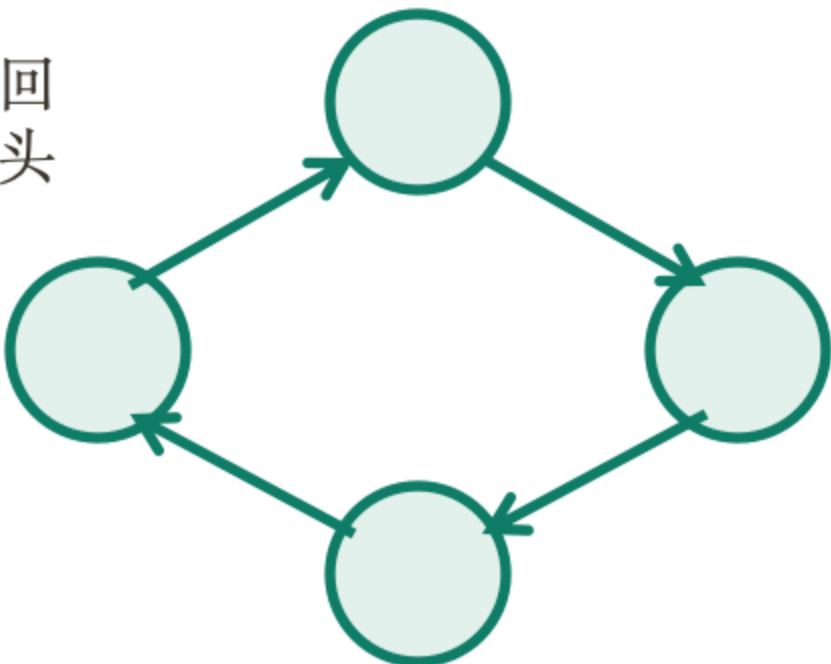
双向链表



每个节点同时保存前驱与后继指针，可双向遍历，但占用更多空间。

循环链表

尾节点回头指向头节点



对比项	数组	链表
随机访问（找第 k 个）	$O(1)$ 报下标直达	$O(n)$ 从头数起
插入 / 删除	$O(n)$ 挪动一大片	$O(1)$ 只改箭头
内存	连续一整块	分散、靠指针串

★ 初赛速记

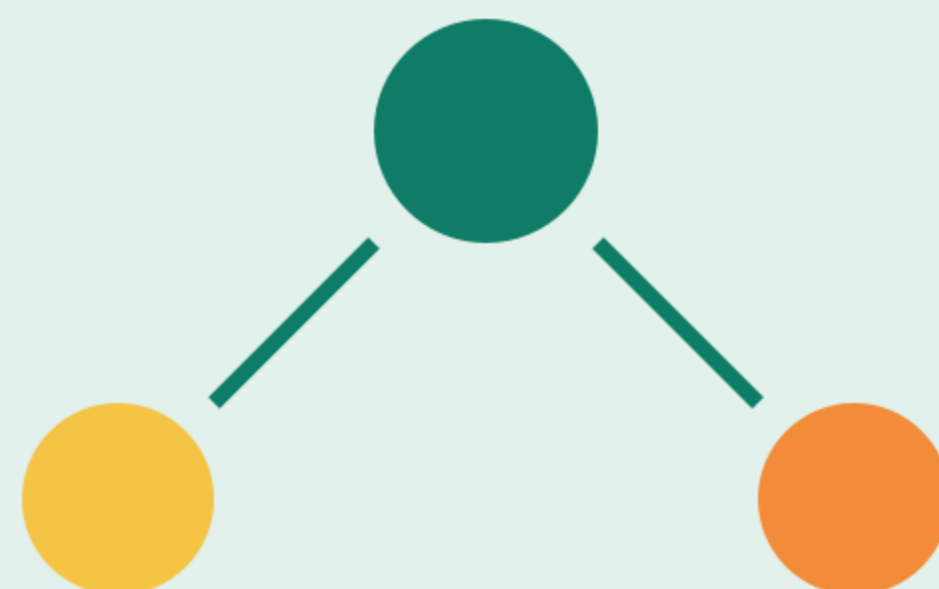
- 链表不支持随机访问
- 插入/删除为 $O(1)$ （已知前驱时）
- 双向链表以空间换双向遍历能力
- 循环链表结束条件：回到头节点

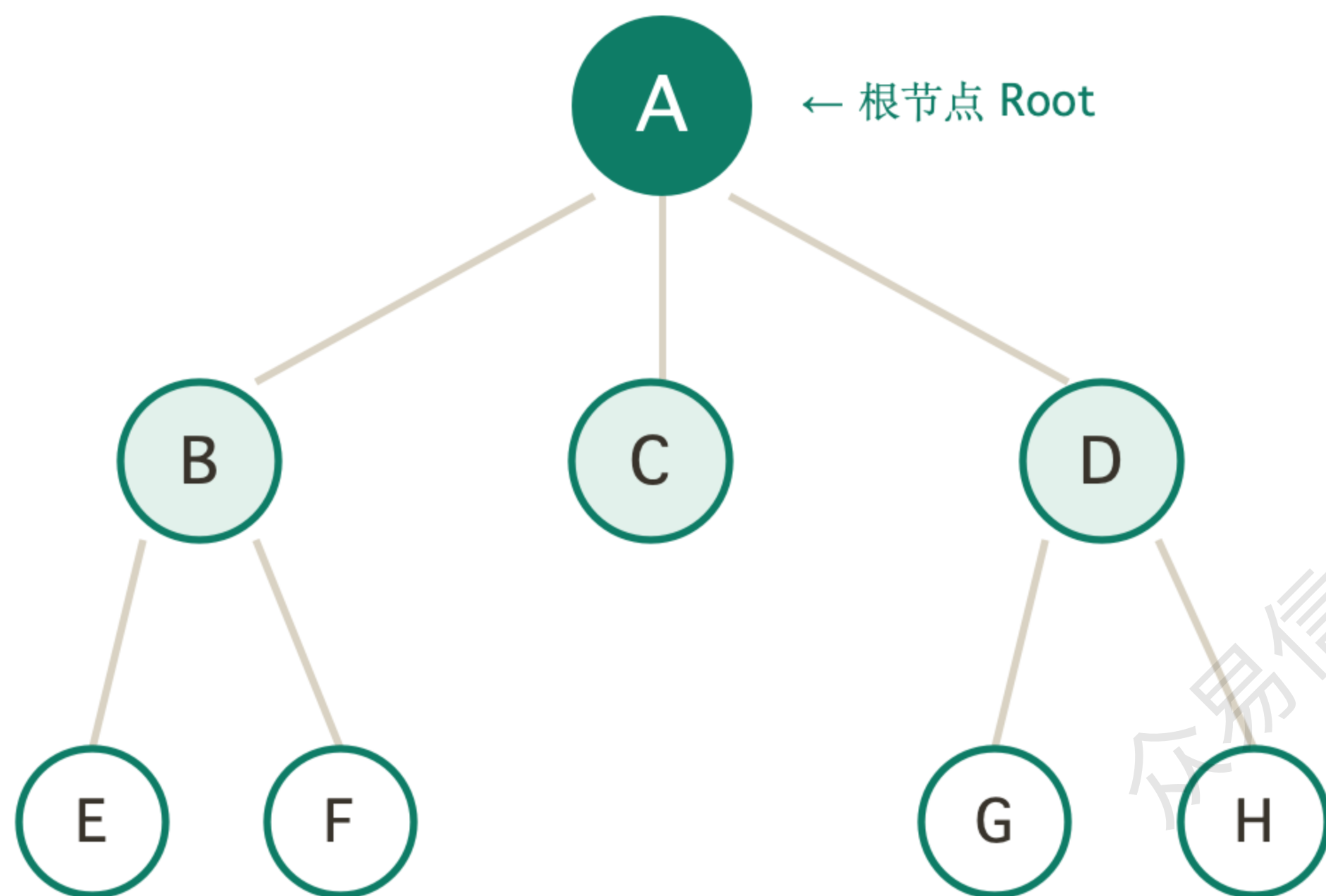
02

第 2 章 · CHAPTER 02

树与二叉树

从线性结构到层级结构：树的定义、术语、性质与遍历。





计算机中的树按"根在上、叶子在下"绘制，是一种典型的层级结构。

① **一对多**：一个节点可以有多个孩子，但每个非根节点只有一个双亲。

② **唯一的根**：根节点没有双亲，是整棵树的入口。

③ **无环**：从任一节点出发沿边走，不可能回到其自身。

★ **重要性质**： n 个节点的树恰有

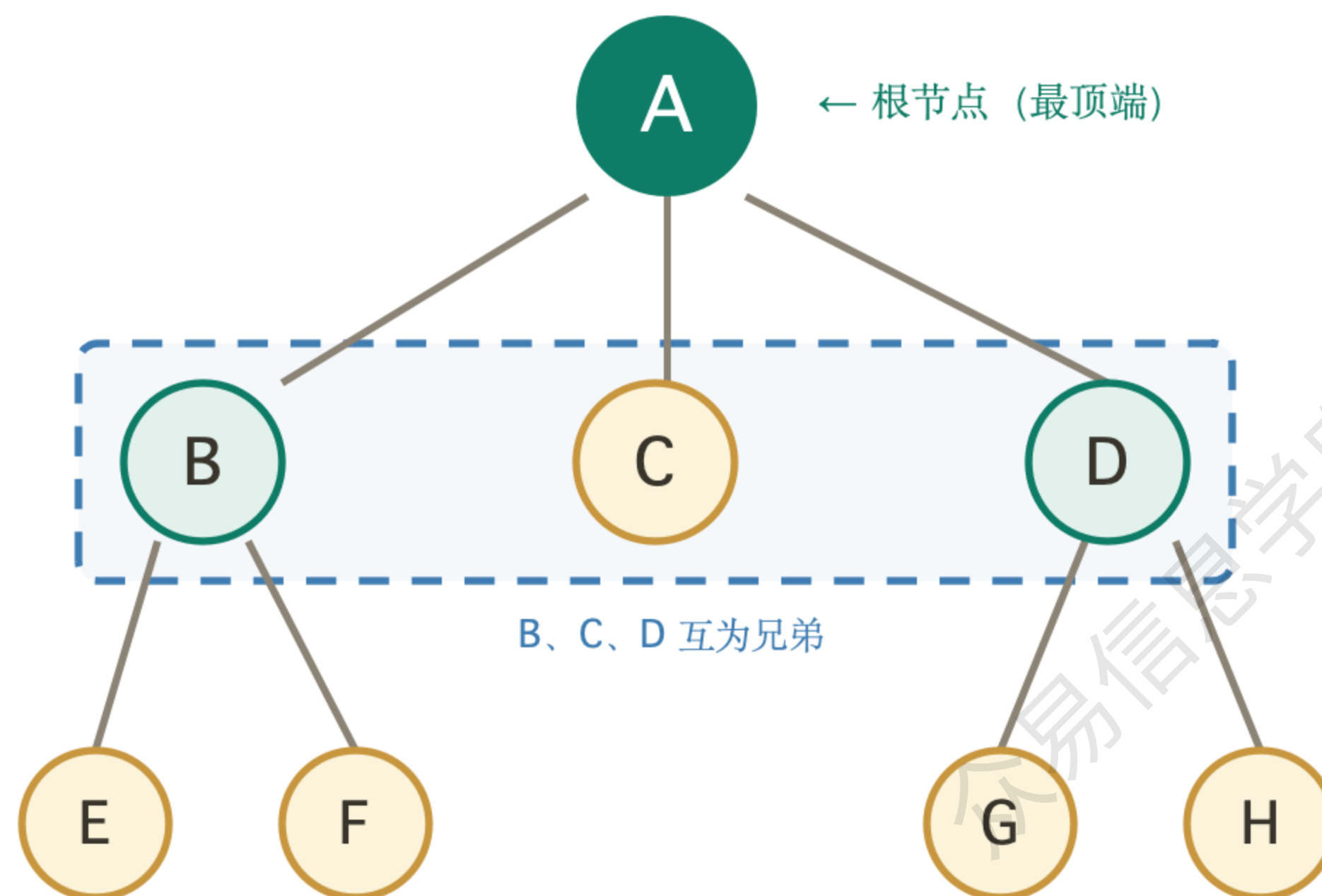
$n - 1$ 条边（每个非根节点有且仅有一个双亲）

上图：8 个节点，7 条边 ✓

第 1 层

第 2 层

第 3 层



黄色 = 叶子节点: C、E、F、G、H (度为 0)

该树的深度 (高度) 为 3 | A 的度为 3, 树的度 = 3

术语汇总

度: 节点的孩子个数。deg(A) = 3

叶子: 度为 0 的节点

内部节点: 非根非叶节点, 如 B、D

双亲 / 兄弟: A 是 B 的双亲; B、C、D 互为兄弟

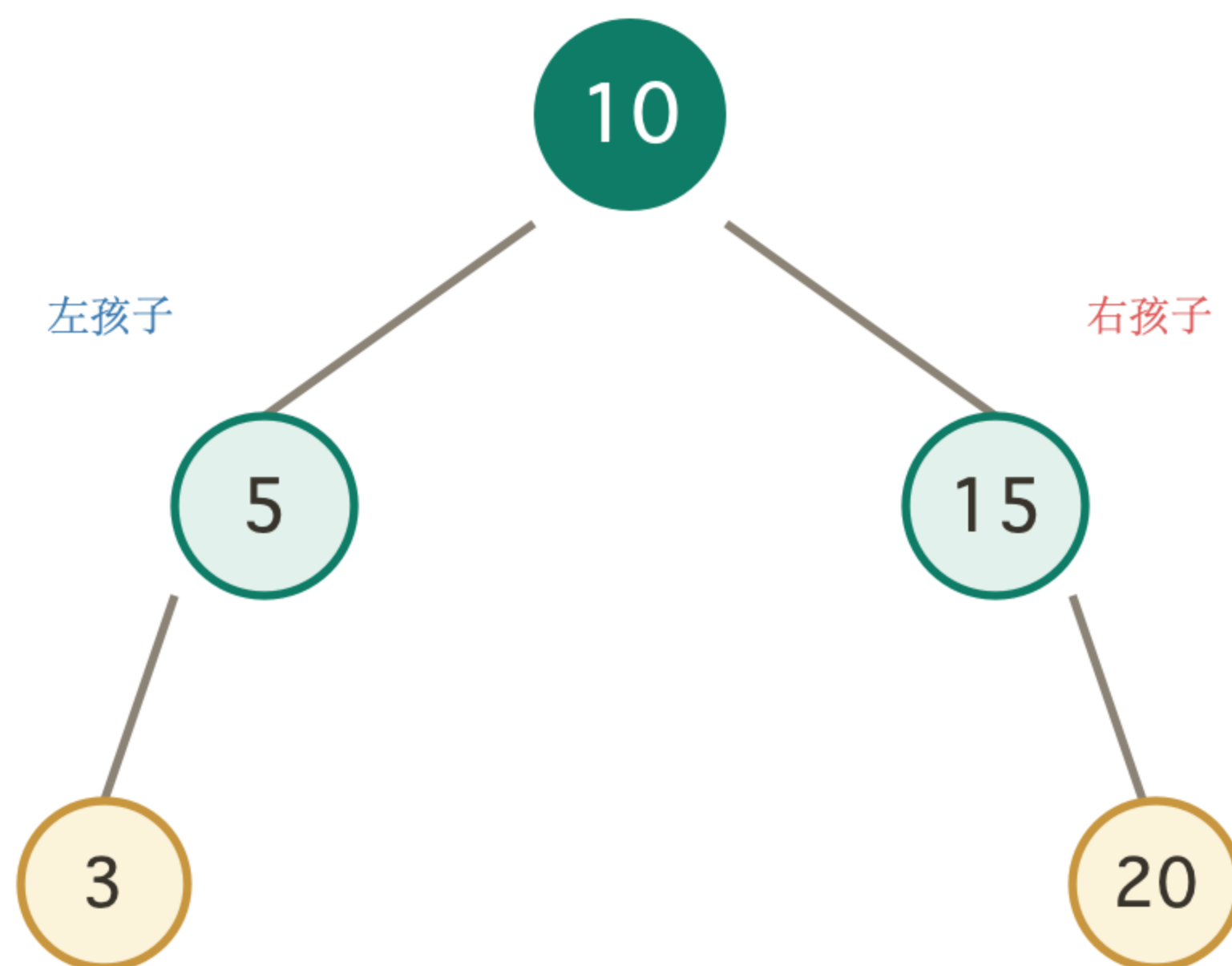
祖先: E 的祖先是 B 和 A

层次: 根为第 1 层, 向下依次递增

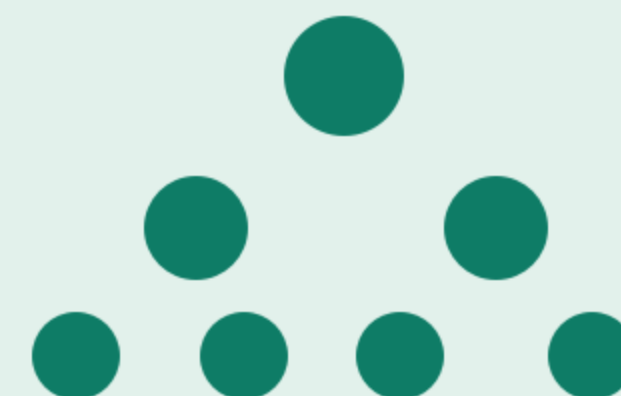
深度: 最大层次 = 3

树的度: 各节点度的最大值 = 3

二叉树：每个节点至多两个孩子，且区分左右



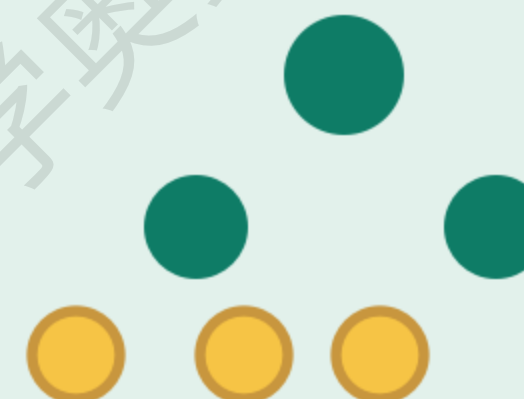
⚠ 即使只有一个孩子，也必须区分左、右。
3 是 5 的左孩子，20 是 15 的右孩子 —— 左右位置不同，即视为不同的二叉树。



满二叉树

除叶子外每个节点都有两个孩子，每层节点数均达到最大。

深度 3 → 恰好 7 个节点



完全二叉树

前 $k-1$ 层全满，最后一层节点靠左连续排列。

满二叉树必为完全二叉树

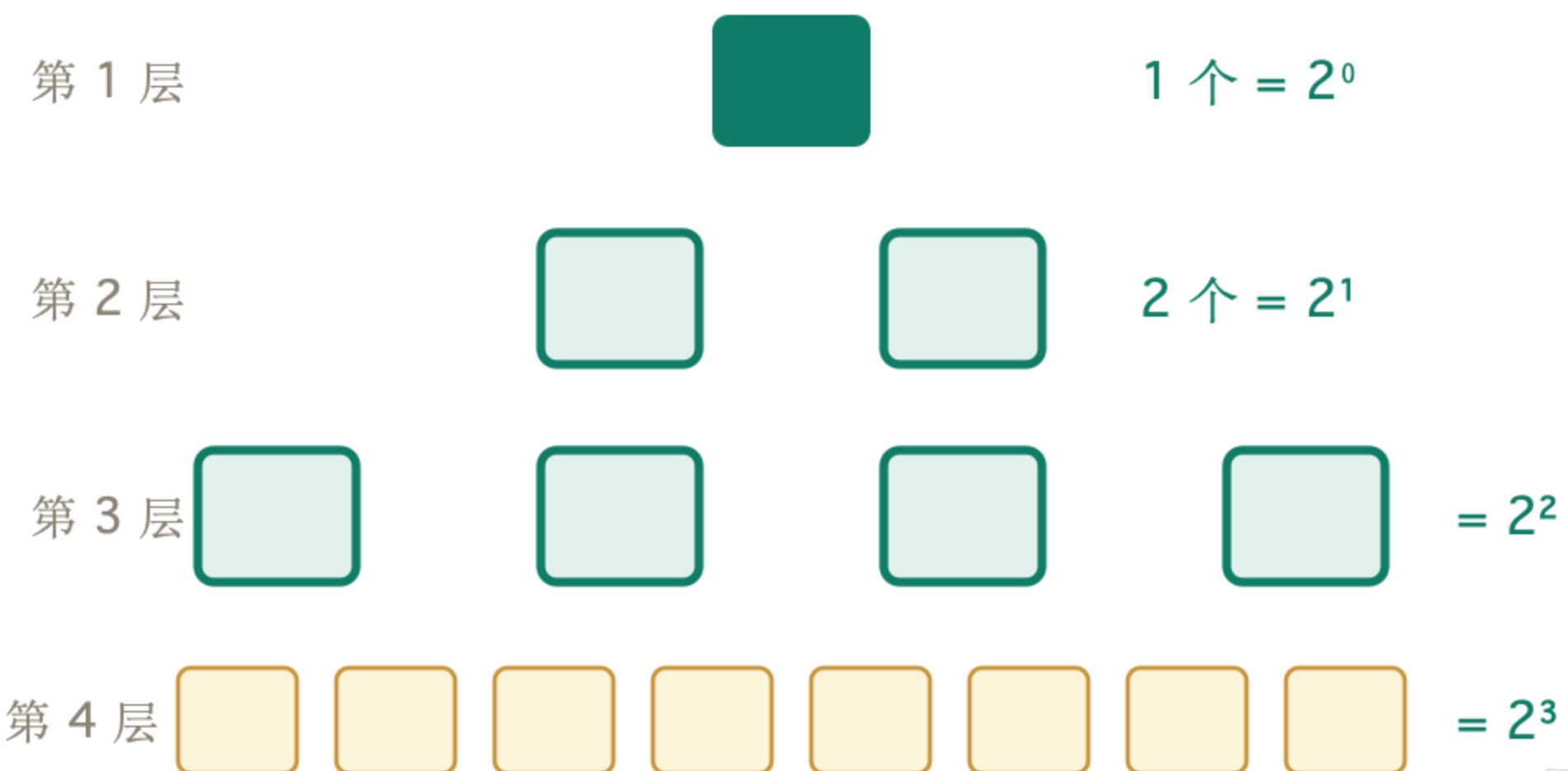


非完全二叉树

最后一层中间存在空位（红虚线），
违反了“靠左连续排列”的要求。

二叉树的三条重要性质

每层的节点数上限：逐层翻倍



$1 \rightarrow 2 \rightarrow 4 \rightarrow 8 \dots$ 每向下一层，容量翻倍，这就是性质①②的由来。

前 4 层全满： $1+2+4+8 = 15 = 2^4 - 1$ ✓

性质① 第 i 层至多有 2^{i-1} 个节点

约定根为第 1 层；每向下一层，上限 $\times 2$ 。

性质② 深度为 k 的二叉树至多有 $2^k - 1$ 个节点

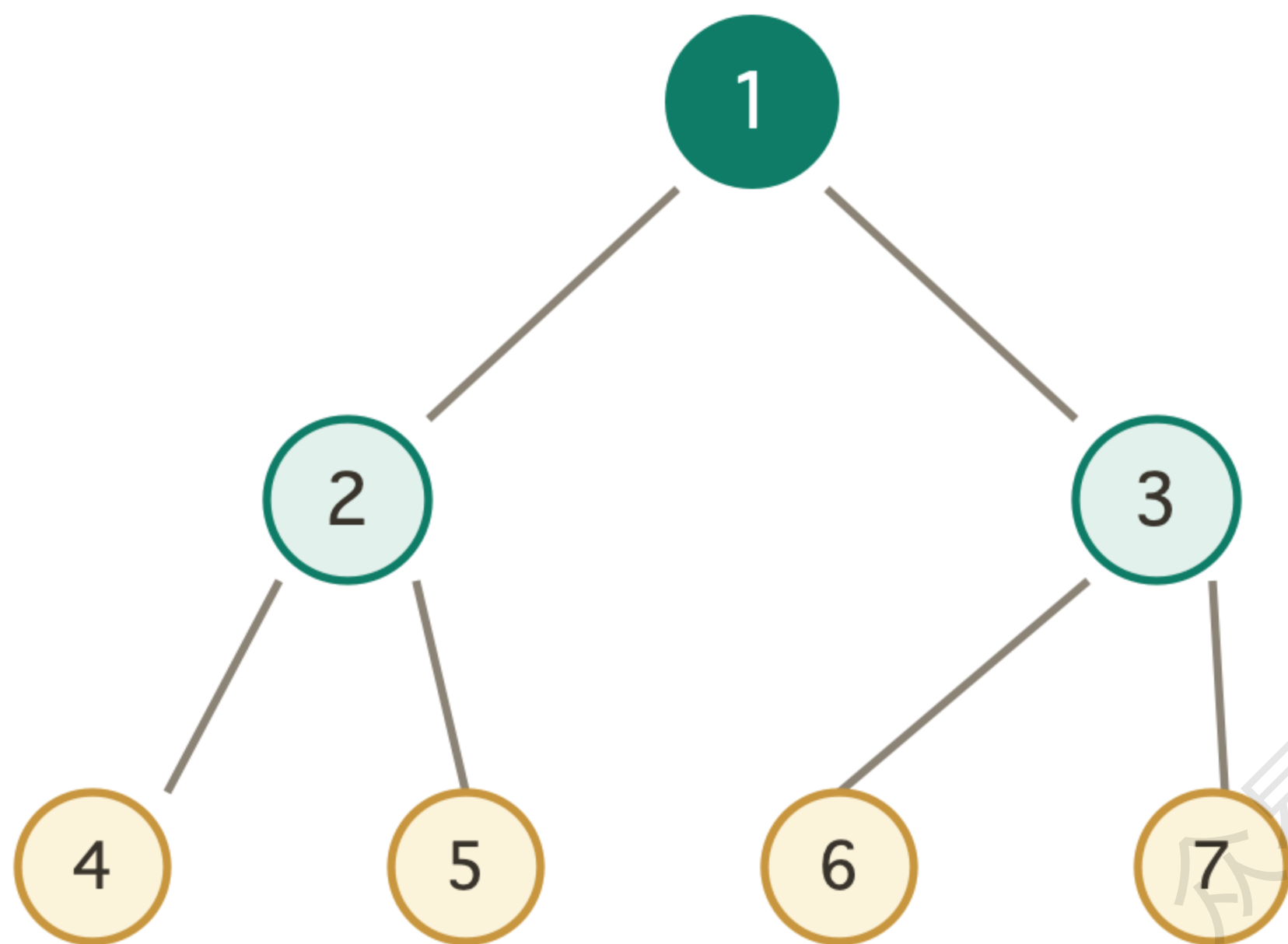
取等号时即满二叉树；例： $k = 3 \rightarrow 2^3 - 1 = 7$ 。

性质③ ☆ 任意二叉树： $n_0 = n_2 + 1$

叶子数 = 度为 2 的节点数 + 1。验证上页： $n_2 = 1$ (节点 10) ,

$n_0 = 2$ (节点 3、20) ✓

完全二叉树的数组存储：编号即下标



↓ 按层次编号存入数组，编号直接作为下标



编号规律（从 1 开始编号）

节点 i 的父节点： $\lfloor i/2 \rfloor$

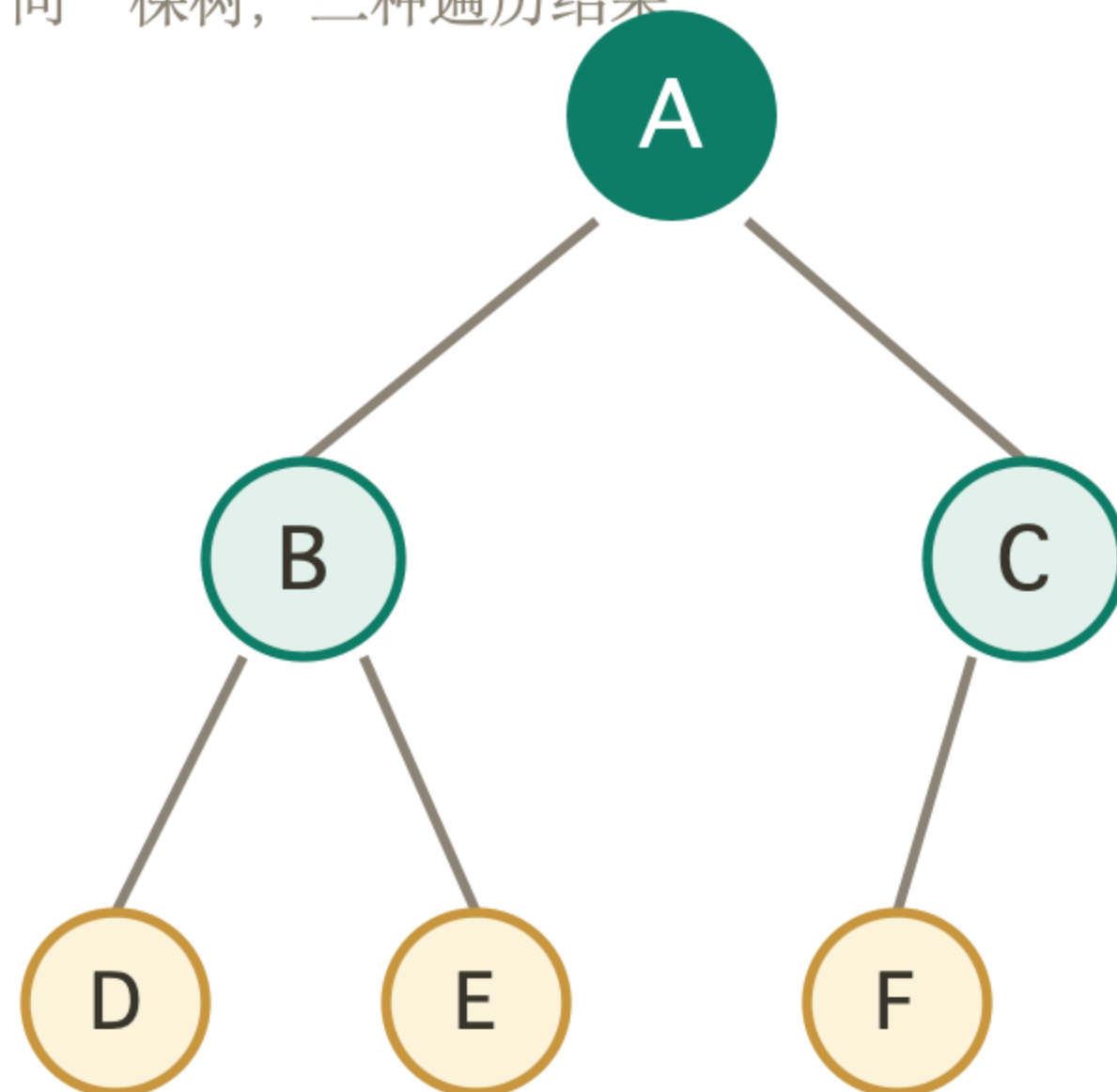
节点 i 的左孩子： $2i$

节点 i 的右孩子： $2i + 1$

验证节点 3：父 = 1，左 = 6，右 = 7
与左图结构一致 ✓

★ 应用：堆（优先队列）就是用数组存储的完全二叉树。

同一棵树，三种遍历结果



遍历：按确定顺序访问每个节点，且每个节点恰好访问一次。
三种方式的区别仅在于根节点的访问时机。

前序遍历：根 → 左 → 右

A B D E C F

先访问根，再依次递归遍历左子树、右子树

中序遍历：左 → 根 → 右

D B E A F C

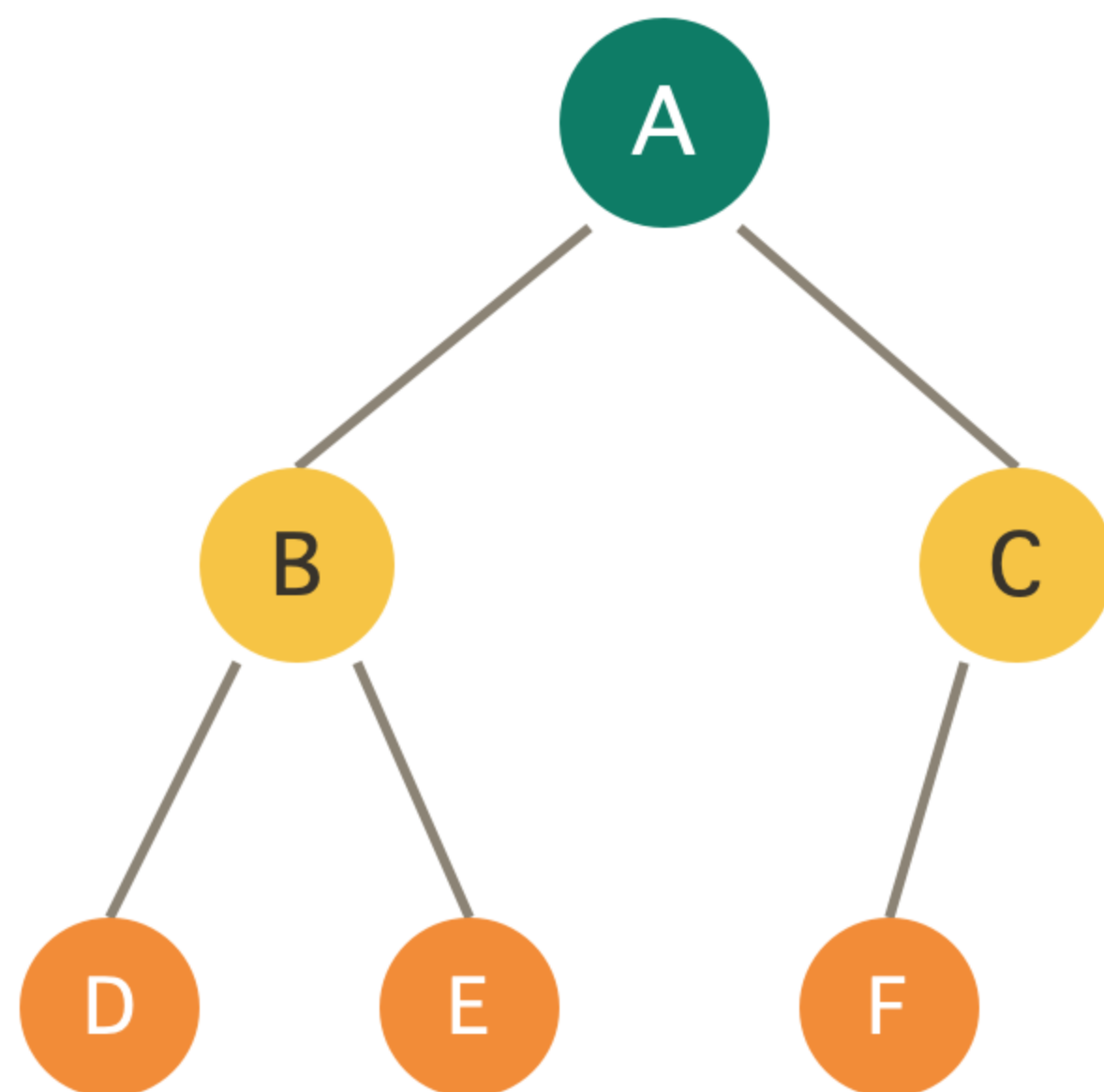
先遍历左子树，访问根，再遍历右子树

后序遍历：左 → 右 → 根

D E B F C A

先遍历左、右子树，最后访问根

层次遍历：自上而下，逐层从左到右



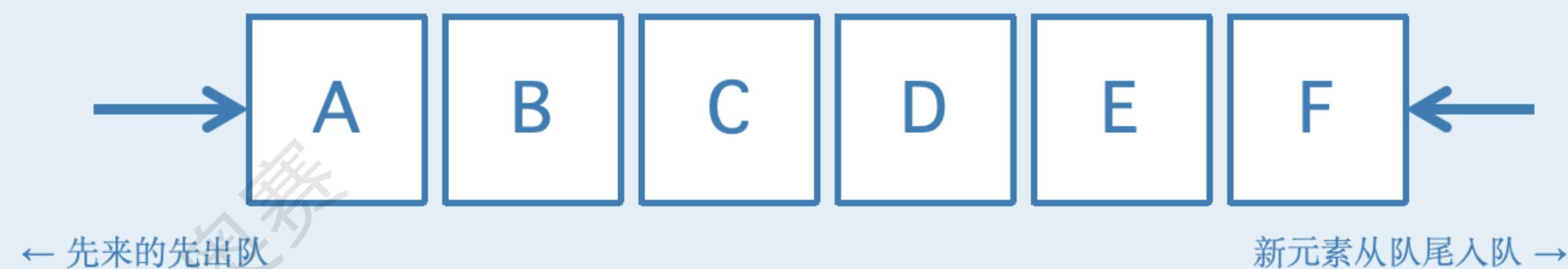
● 第1层 ● 第2层 ● 第3层

规则：自上而下，每一层从左到右。

结果：**A B C D E F**

访问顺序与节点所在的层完全一致。

实现工具：队列（先进先出）



执行过程：

- ① 根 A 入队；A 出队时，其孩子 B、C 依次入队
- ② B 出队，D、E 入队；C 出队，F 入队
- ③ D、E、F 依次出队 —— 恰好逐层访问

由遍历序列还原二叉树（初赛高频考点）

① 已知条件

前序（根左右）：
G B A D C F E H

中序（左根右）：
A B C D G E F H

② 确定根节点

前序序列的**首元素**必为根
→ 根 = **G**

在中序序列中定位 G：
左侧 ABCD 为左子树
右侧 EFH 为右子树

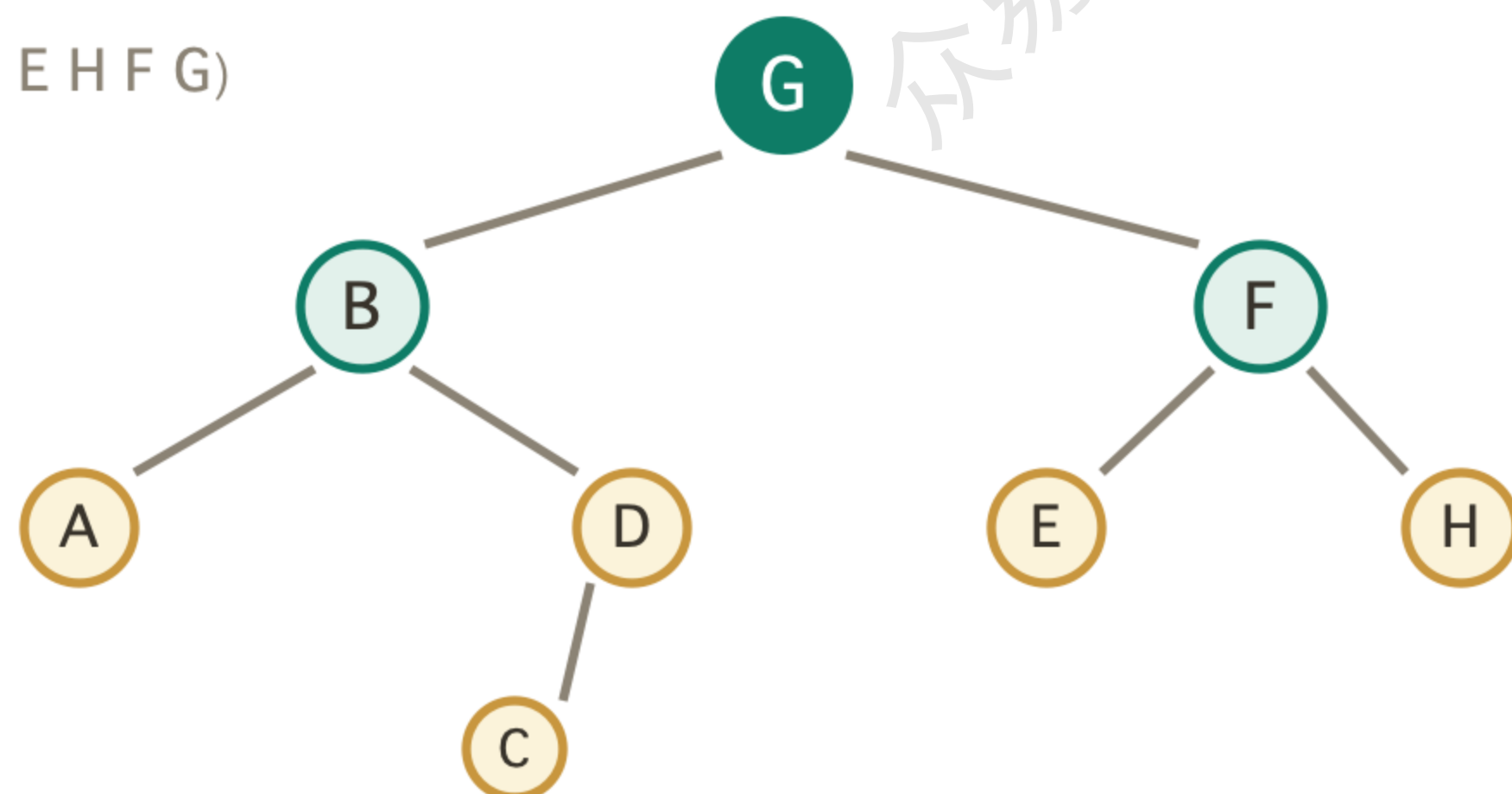
③ 递归划分子问题

左子树：前序 **BADC** + 中序 **ABCD** → 根为 **B**

右子树：前序 **FEH** + 中序 **EFH** → 根为 **F**
对每棵子树重复上述过程.....

整棵二叉树即可唯一还原

还原结果（后序：A C D B E H F G）



★ 能否唯一确定？

前序 + 中序 ☒ 可以

后序 + 中序 ☒ 可以

前序 + 后序 ☒ 不能

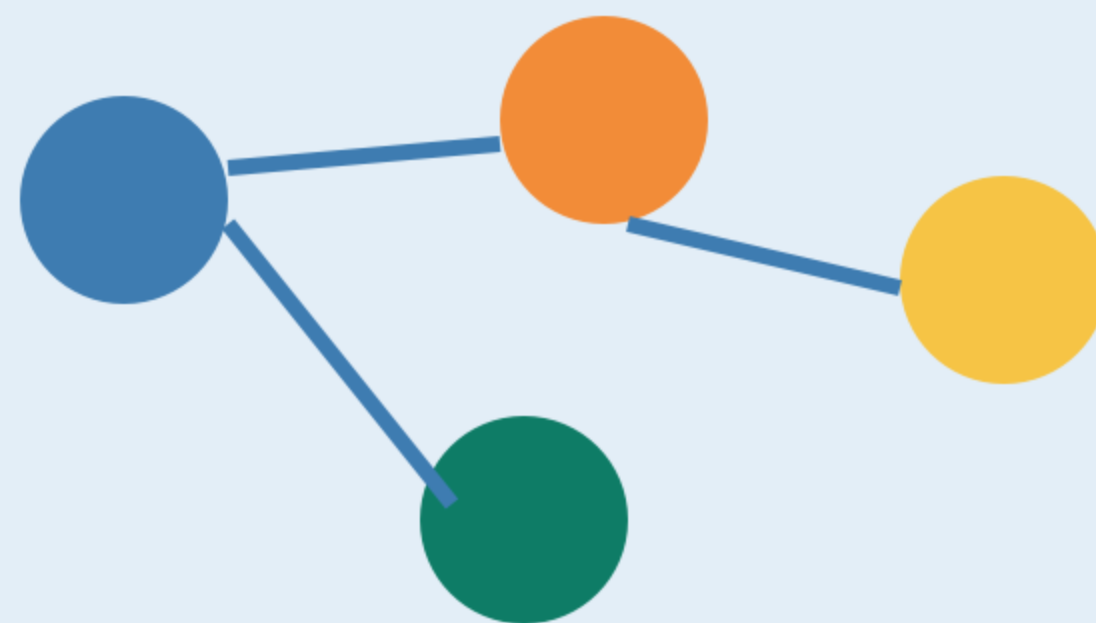
（满二叉树除外）

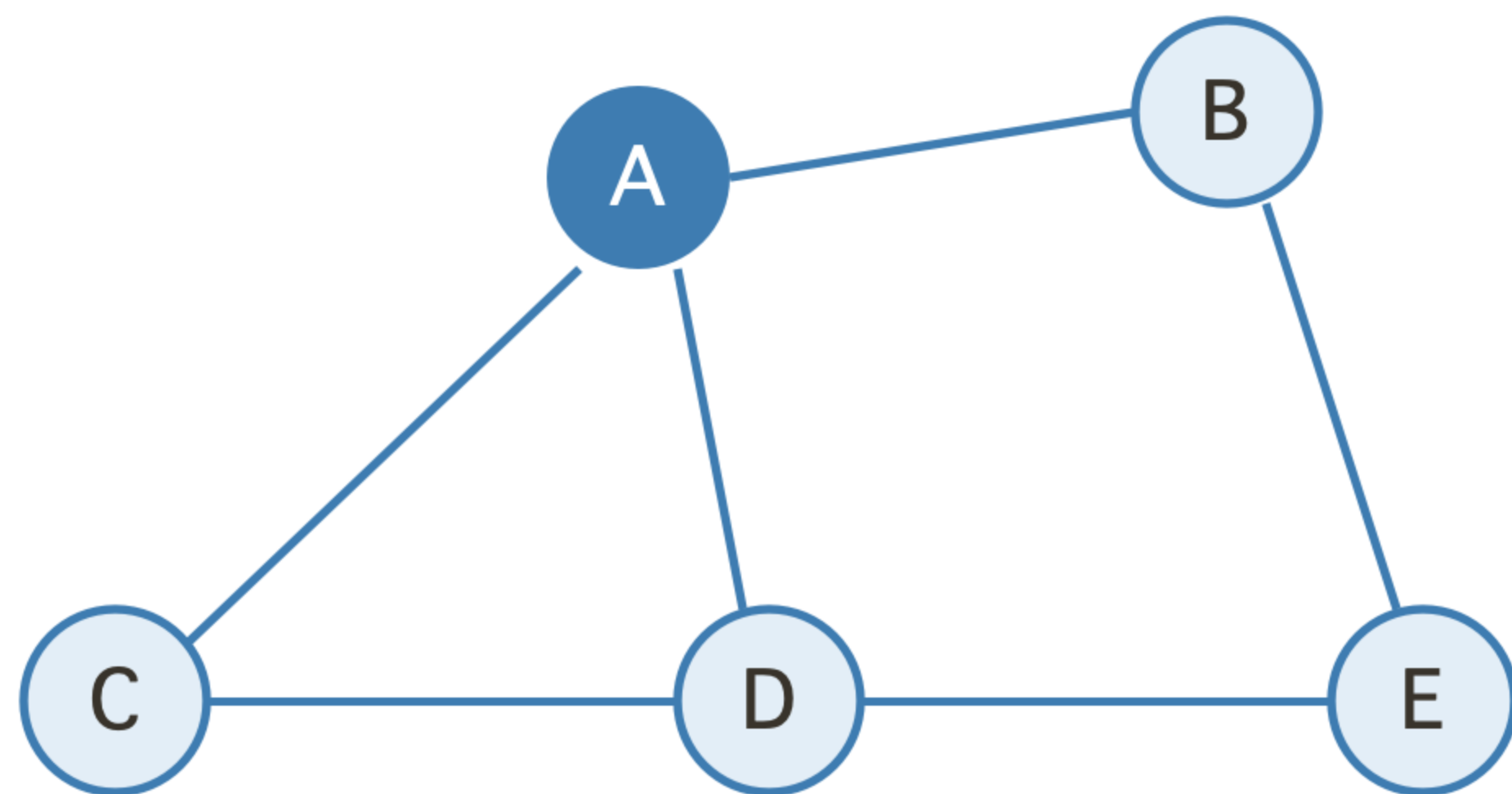
03

第 3 章 · CHAPTER 03

图

研究对象之间任意的连接关系：顶点、边、存储与遍历。





该图包含 5 个顶点、6 条边；A 的邻接点为 B、C、D。

路径：A → B → E 是 A 到 E 的一条路径（长度 2）；
A → C → D → E 同样可达（长度 3）。同一对顶点间可有多条路径。

顶点 (Vertex)：图中的基本对象。

边 (Edge)：顶点之间的连接关系。

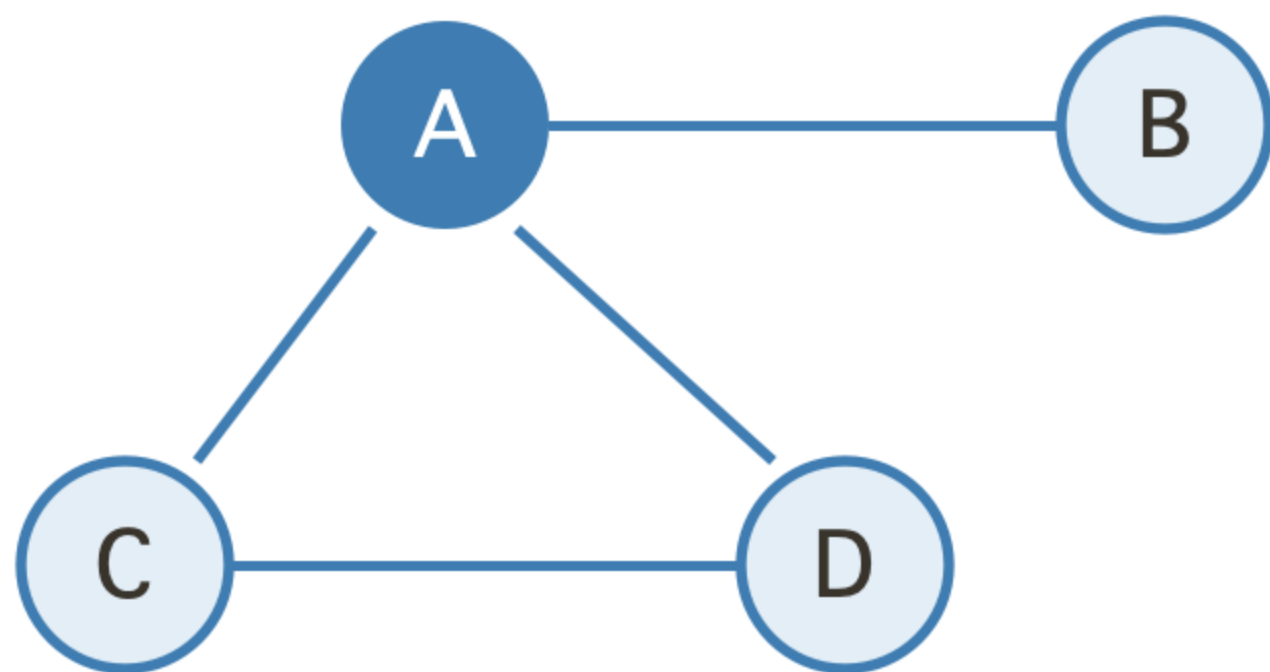
树是特殊的图：连通且无环。

一般的图允许存在环，也允许不连通。

图的典型应用场景

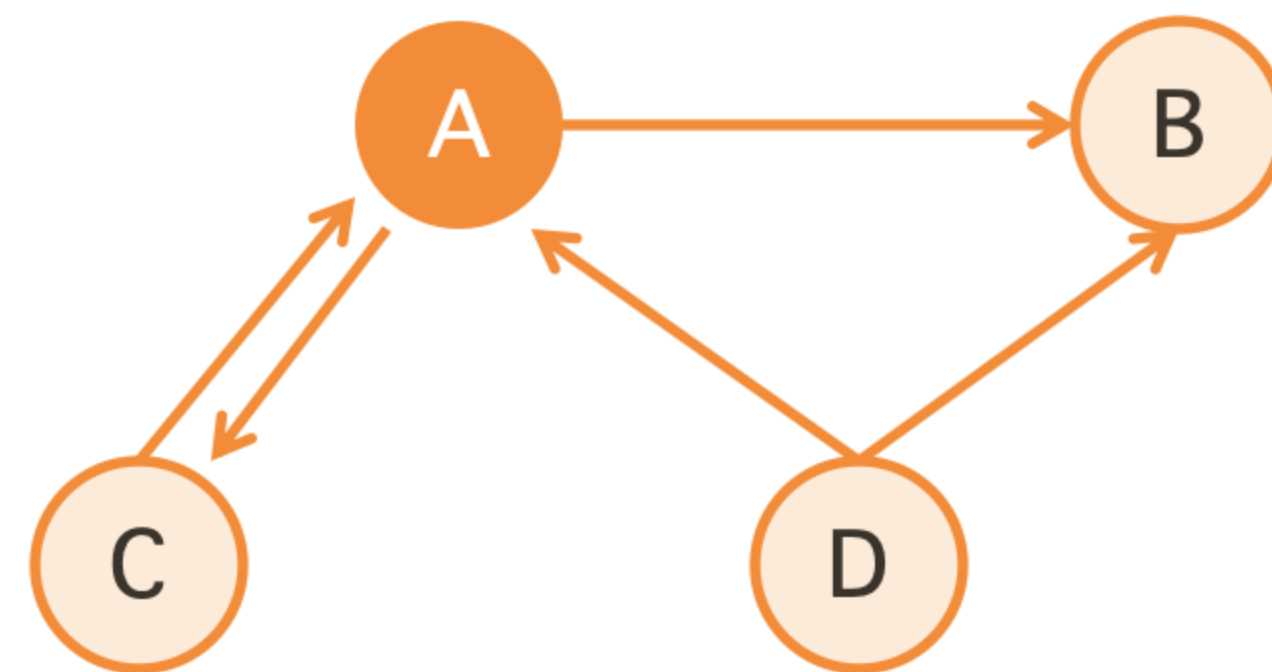
- 交通网络：站点为顶点，线路为边
- 社交网络：用户为顶点，好友关系为边
- 游戏地图：区域为顶点，通路为边
- 网页结构：页面为顶点，超链接为边

无向图：边没有方向



度 = 相连的边数: $\deg(A)=3$, $\deg(B)=1$, $\deg(C)=2$, $\deg(D)=2$

有向图：边有方向



$A \rightarrow B$ 与 $B \rightarrow A$ 是两条不同的边，只能沿箭头方向通行。

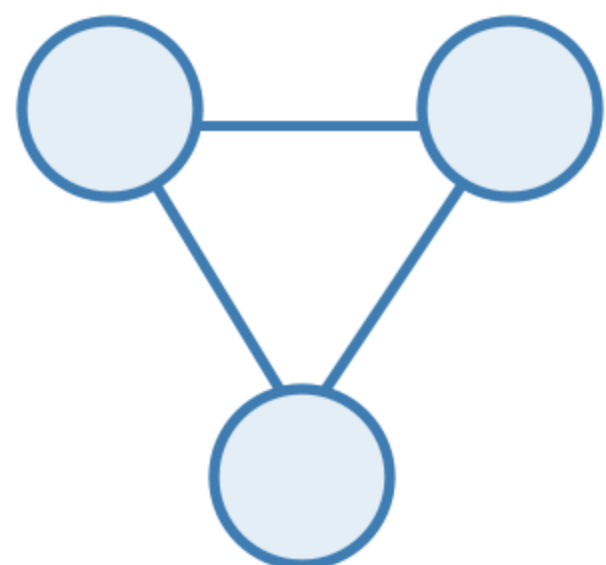
有向图节点	A	B	C	D
入度 (指向它的边数)	2	2	1	0
出度 (由它发出的边数)	2	0	1	2

★ 重要结论

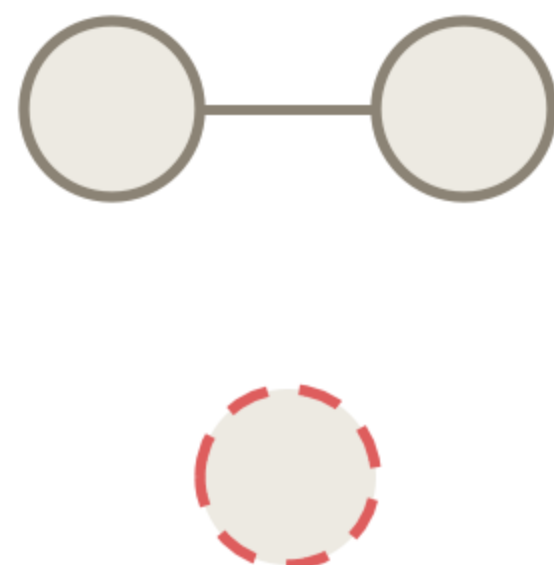
入度之和 = 出度之和 = 边数

验证: $2+2+1+0 = 5 = 5$ 条边 ✓

连通 vs 非连通

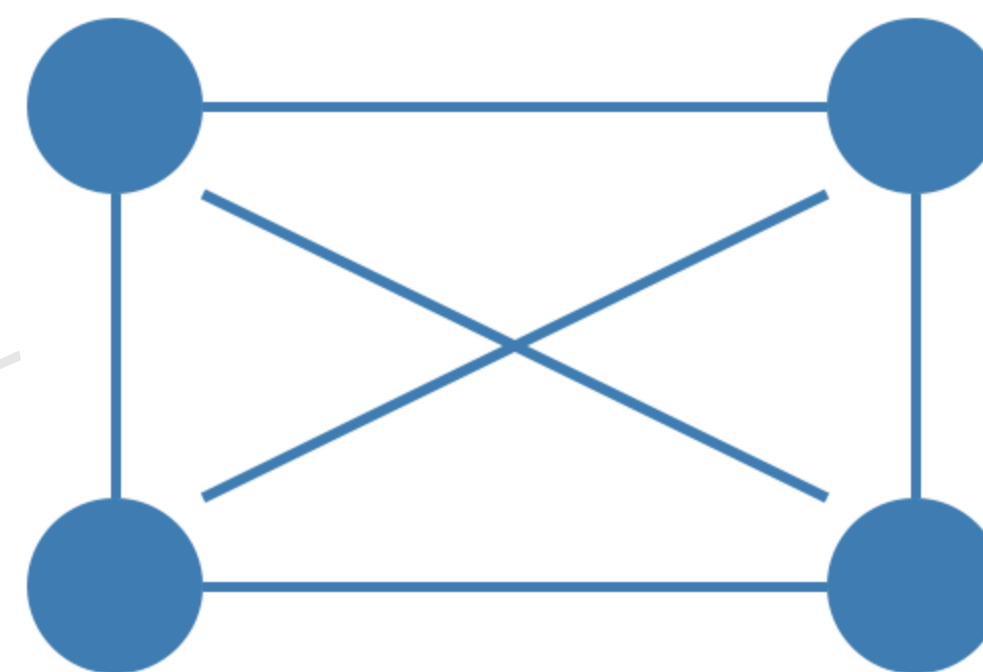


连通图 ✓ 任意两点可达



非连通：存在孤立点 (2 个连通分量)

完全图：每对顶点都直接相连



K_4 : 4 个顶点、6 条边

n 个顶点的无向完全图：

$\frac{n(n-1)}{2}$ 条边

验证： $4 \times 3 \div 2 = 6$ ✓

握手定理

无向图中：所有顶点的度数之和 = 边数 $\times 2$

原因：每条边有两个端点，统计度数时，

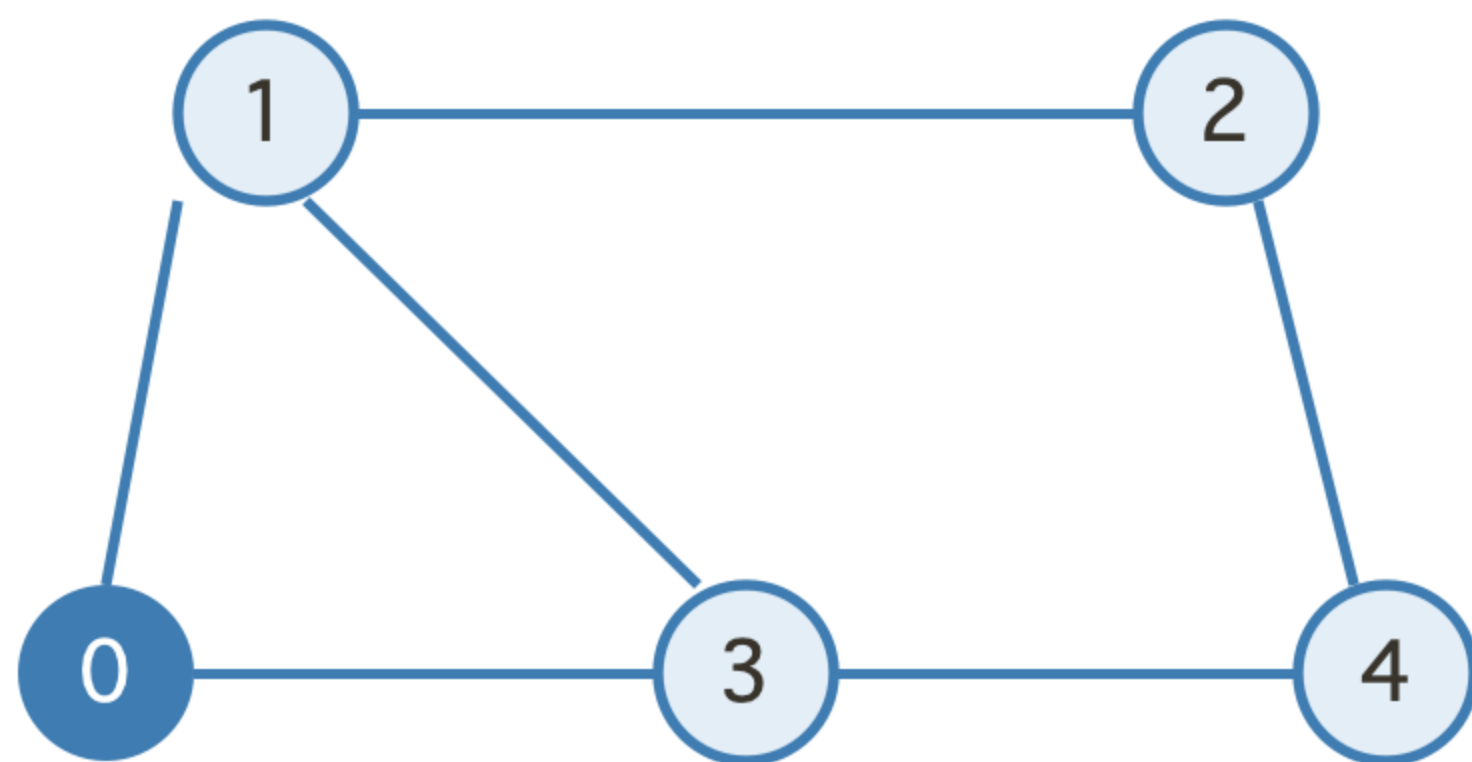
每条边恰好被计算两次。

验证（左上图）：

$$3 + 1 + 2 + 2 = 8$$

$$\text{边数} \times 2 = 4 \times 2 = 8 \checkmark$$

推论：度数之和必为偶数



5 个顶点、5 条边的无向图。

邻接矩阵：用 $n \times n$ 的二维数组记录
顶点对之间是否有边 —— 有边记 1，
无边记 0（见右表）。

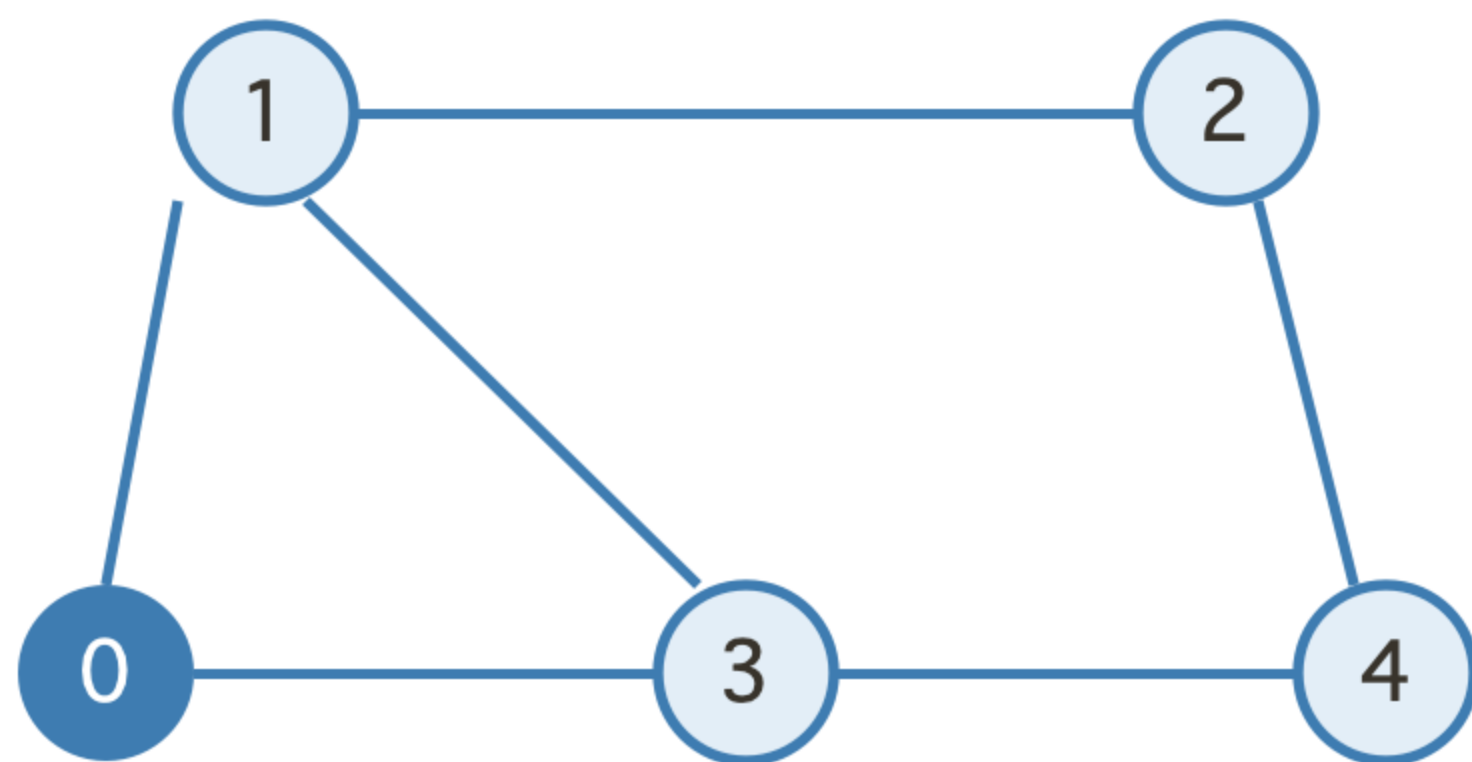
	0	1	2	3	4
0	0	1	0	1	0
1	1	0	1	0	0
2	0	1	0	0	1
3	1	0	0	0	1
4	0	0	1	1	0

高亮格 = 有边
例：第 0 行第
1 列为 1，表
示 0 与 1 相连
✓

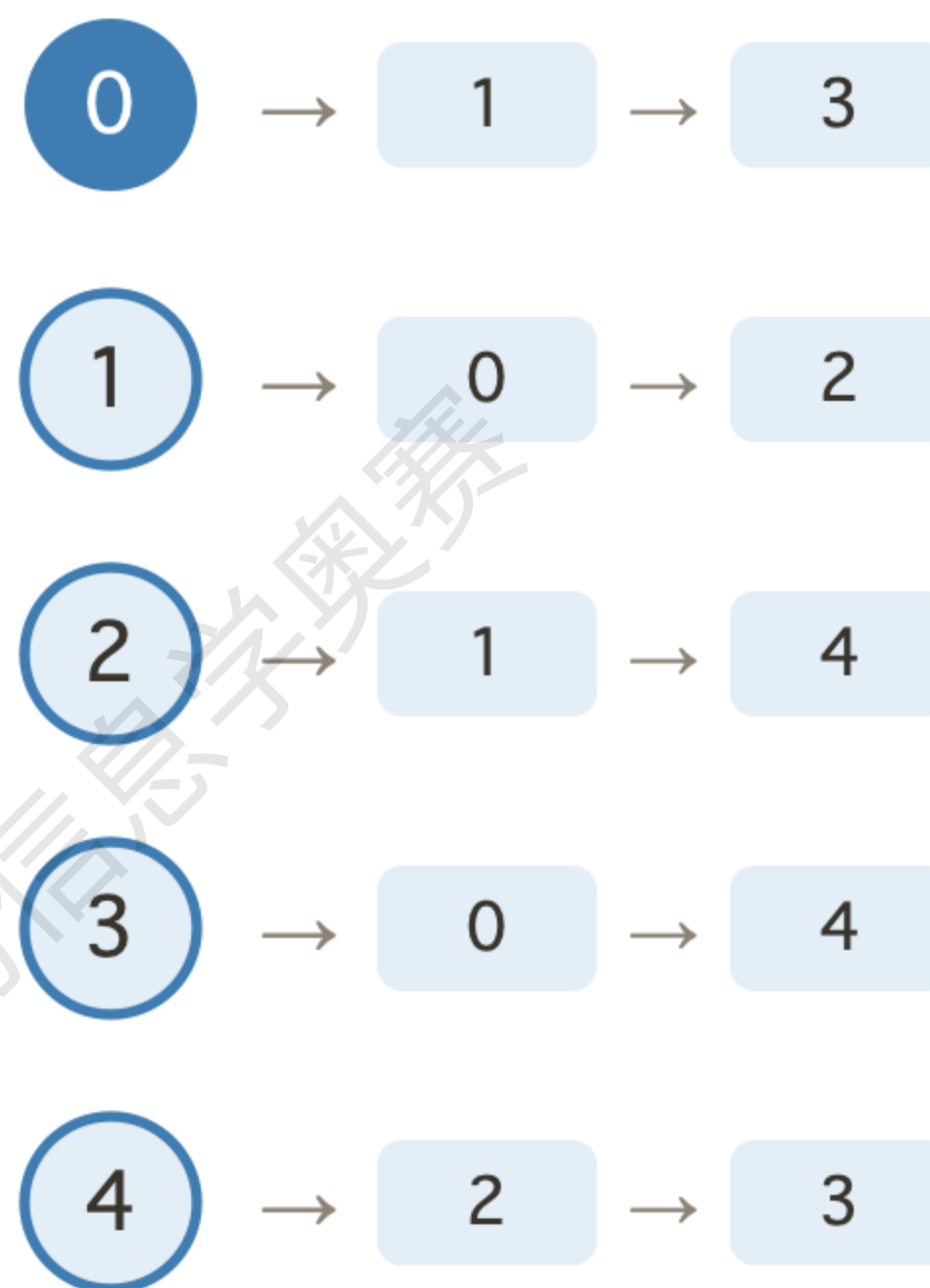
无向图的邻接
矩阵关于主对
角线对称

✓ 判断任意两点间是否有边，一次查询即可： $O(1)$

✗ 空间 $O(n^2)$ ：稀疏图中大量元素为 0，浪费空间



同一张图的另一种存储思路：
不记录所有顶点对，而是为每个顶点
维护一个邻接顶点列表（见右图）。



列表长度 = 该顶点的度

✅ 只存储实际存在的边：
空间 $O(n + e)$ ，适合稀疏图

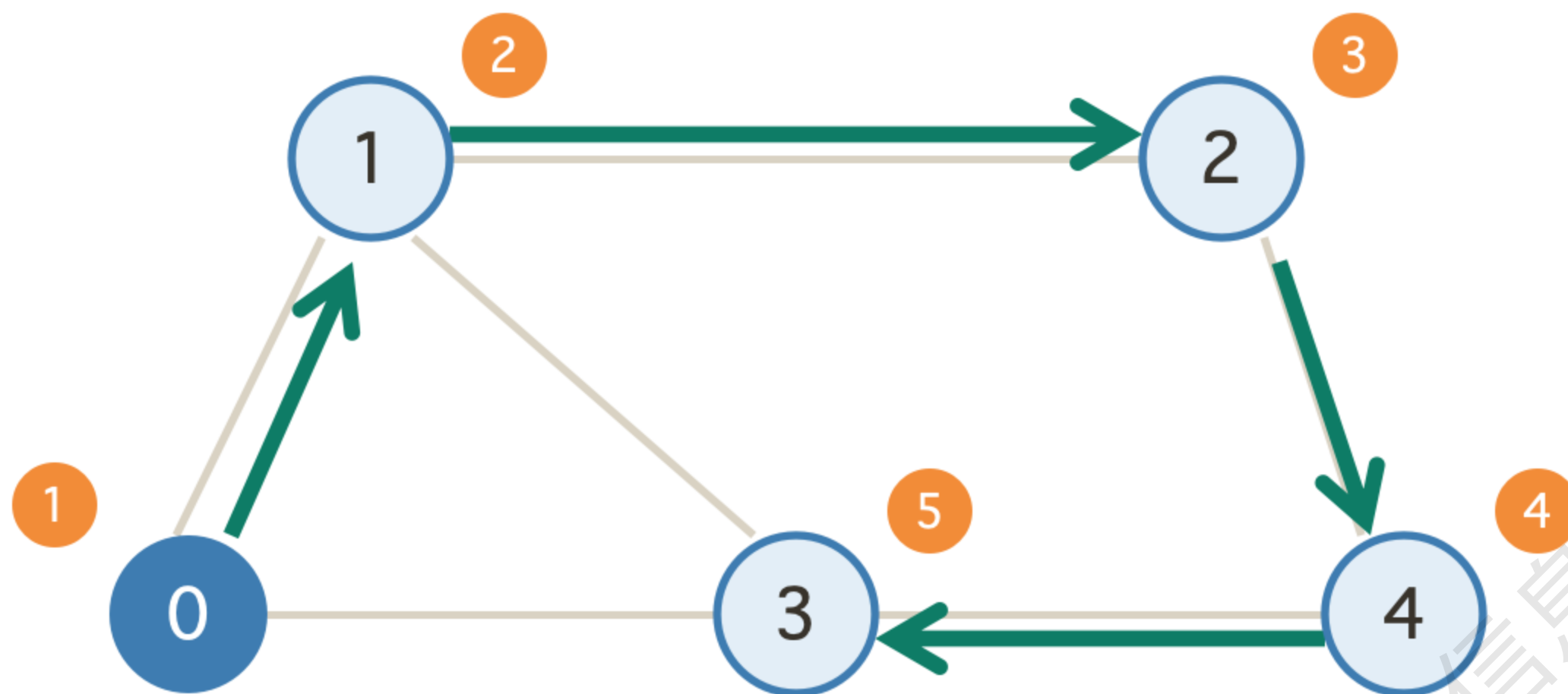
❌ 判断两点间是否有边，
需遍历对应列表： $O(k)$

进阶：竞赛中常用数组实现的"链式前
向星"，复赛阶段再学习。

★ 稀疏图适合邻接表；需频繁查询点对关系适合邻接矩阵。邻接表中的每个列表，本质上就是一条链表。

深度优先搜索 DFS: 纵深优先, 回溯展开

03 图



从 0 出发的访问顺序: $0 \rightarrow 1 \rightarrow 2 \rightarrow 4 \rightarrow 3$

从 0 出发: 先访问邻接点 1, 沿纵深到 2、4, 最后访问 3

核心思想

沿一条路径不断深入, 无法继续时回溯到最近的分叉点, 改走尚未访问的分支 (类似迷宫搜索)。

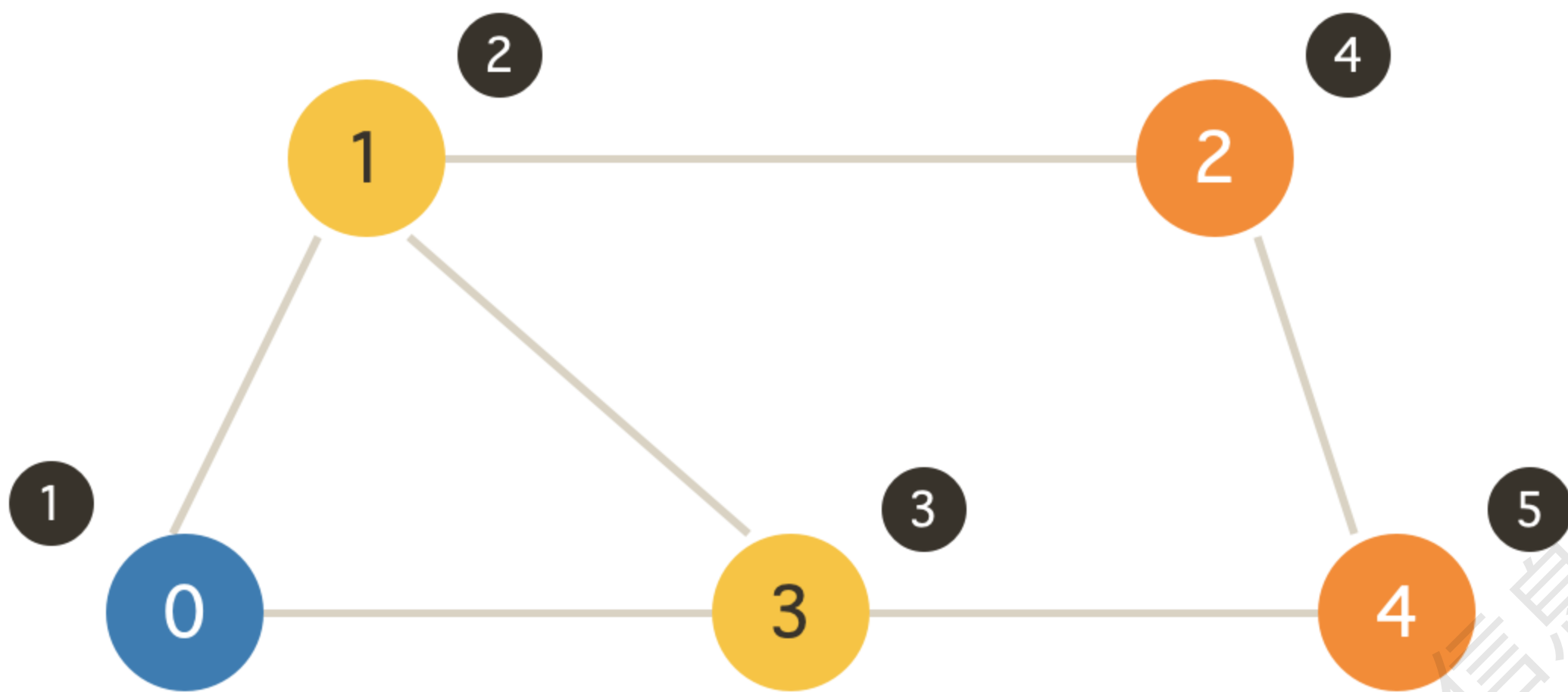
三个要点

- ① 尽可能纵深访问, 必要时回溯
- ② 用 **visited** 数组标记已访问顶点, 避免重复
- ③ 借助**栈**实现 (递归本质上是系统调用栈)

★ 典型应用: 路径枚举、环检测、连通性判断

关键词: 纵深、回溯、栈

广度优先搜索 BFS：按距离逐层扩展



从 0 出发的访问顺序：0 → 1 → 3 → 2 → 4

蓝 = 起点；黄 = 距离起点 1；橙 = 距离起点 2

核心思想

按到起点的距离由近及远逐层扩展：先访问所有距离为 1 的顶点，再访问距离为 2 的顶点。

实现工具：**队列**（先进先出），先发现的顶点先扩展。

	DFS 深度优先	BFS 广度优先
实现工具	栈 / 递归	队列
遍历顺序	纵深优先、回溯	逐层扩展
典型应用	路径枚举、连通性	无权图最短路径

指针

- 指针的值是内存地址，不是数据本身
- **&** 取地址，***** 解引用
- 数组名通常退化为首元素指针

链表

- 节点 = 数据域 + 指针域
- 插入/删除 **O(1)**: 仅修改指针
- 访问 **O(n)**: 不支持随机访问

树

- n 个节点恰有 $n - 1$ 条边
- 唯一根、无环、一对多
- 度 = 孩子数; 深度 = 最大层次

二叉树

- 第 i 层最多 2^{i-1} 个节点
- 深度 k 最多 $2^k - 1$ 个节点
- $n_0 = n_2 + 1$ (叶子数 = 度为2的节点数 + 1)
- 完全二叉树: 父 $\lfloor i/2 \rfloor$, 左 $2i$, 右 $2i + 1$

遍历

- 前序 根左右 | 中序 左根右
- 后序 左右根 | 层次遍历借助队列
- 前序+中序 / 后序+中序可唯一确定
- 前序+后序不能 (满二叉树除外)

图

- 握手定理: 度数之和 = 边数 $\times 2$
- 邻接矩阵 $O(n^2)$; 邻接表 $O(n+e)$
- DFS 借助栈; BFS 借助队列, 可求无权最短路

下一讲：用 C++ 实现它们

本讲内容：指针、链表、树、二叉树、图——概念、性质、存储与遍历



课后练习

画出一棵含 6 个节点的二叉树，
写出它的前序、中序、后序遍历序列，
并验证性质 $n_0 = n_2 + 1$ 是否成立。

本讲结束